

SQL Revision Notes

NESTED SUBQUERY VS. CORRELATED SUBQUERY VS. JOIN

1. Core Scenario Setup

We evaluate three core SQL approaches using a single baseline dataset. Goal: **Find complete details of employees who work in any department.**

EMP — Employee Details

E_ID	EName
1	A
2	B
3	C
4	D
5	E

DEPT — Department Details

Dept_No	Dept_Name	E_ID
D1	IT	1
D2	HR	2
D3	Marketing	3

Expected Output (Rows 1, 2, 3): Employees 1 (A), 2 (B), and 3 (C) match departments. Employees 4 (D) and 5 (E) have no department assigned and are excluded.

2. Non-Correlated Nested Subquery BOTTOM-UP

The inner query is **completely independent** of the outer query and runs exactly **once**.

```
SELECT *
FROM Emp
WHERE E_ID IN (
    SELECT E_ID
    FROM Dept
);
```

- Execute Inner Query First:** `SELECT E_ID FROM Dept` → Returns list (1, 2, 3).
- Execute Outer Query:** Evaluates `SELECT * FROM Emp WHERE E_ID IN (1, 2, 3)`.

EMP Row	E_ID	In (1, 2, 3)?	Evaluation Result
1 A	1	Yes ✓	Include Row
2 B	2	Yes ✓	Include Row
3 C	3	Yes ✓	Include Row
4 D	4	No ✗	Exclude Row
5 E	5	No ✗	Exclude Row

Exam Point: '=' vs 'IN'

Use = only when the subquery is guaranteed to return a single scalar value. Use IN when the subquery returns multiple values.

🧠 Memory Trick

- 1 Value:** Use =
- Multiple Values:** Use IN

3. Correlated Subquery TOP-DOWN

The inner query references `Emp.E_ID` from the outer query. It executes repeatedly, **once for every outer row**.

```
SELECT *
FROM Emp
WHERE EXISTS (
    SELECT E_ID
    FROM Dept
    WHERE Emp.E_ID = Dept.E_ID
);
```

- 1 Row 1 (1, A): Subquery checks 1 = Dept.E_ID → Match found (D1) → **TRUE** ✓
- 2 Row 2 (2, B): Subquery checks 2 = Dept.E_ID → Match found (D2) → **TRUE** ✓
- 3 Row 3 (3, C): Subquery checks 3 = Dept.E_ID → Match found (D3) → **TRUE** ✓
- 4 Row 4 (4, D) & Row 5 (5, E): Subquery checks IDs 4 & 5 → No matches → **FALSE** ✗

4. JOIN Approach RELATIONAL COMBINE

Combines tables relationally. Conceptually explained as a **Cartesian product followed by condition filtering**.

```
SELECT Emp.*
FROM Emp
JOIN Dept ON Emp.E_ID = Dept.E_ID;
```

Conceptual Execution Breakdown:

Step 1 (Cross Product): 5 EMP rows × 3 DEPT rows = 15 candidate row combinations.

Step 2 (Filter): Apply `Emp.E_ID = Dept.E_ID` → Keeps matches (1↔1, 2↔2, 3↔3).

5. Mechanics & Execution Direction Comparison

Approach	Execution Path	Subquery Frequency	Typical Operators
Non-Correlated	Bottom-Up (Inner first)	Executed exactly once	IN, NOT IN, =
Correlated	Top-Down (Outer row first)	Re-executed for each outer row	EXISTS, NOT EXISTS
JOIN	Relational Combination	No subquery (Combined relations)	JOIN ... ON, WHERE

6. One Query – Three Syntax Forms

Side-by-Side Implementation Reference

A. Non-Correlated:

```
SELECT * FROM Emp WHERE E_ID IN (SELECT E_ID FROM Dept);
```

B. Correlated:

```
SELECT * FROM Emp WHERE EXISTS (SELECT E_ID FROM Dept WHERE Emp.E_ID = Dept.E_ID);
```

C. JOIN:

```
SELECT Emp.* FROM Emp JOIN Dept ON Emp.E_ID = Dept.E_ID;
```

7. Master Feature Comparison Table

Feature	Non-Correlated Subquery	Correlated Subquery	JOIN
Query inside Query?	✔ Yes	✔ Yes	✗ No
Inner Query Independent?	✔ Yes	✗ No	– N/A
Outer Reference in Inner?	✗ No	✔ Yes	– N/A
Basic Execution Model	Bottom-Up	Top-Down	Combine + Condition
Subquery Executions	1 time total	Repeated per outer row	No separate inner query
Primary Keywords	IN, NOT IN	EXISTS, NOT EXISTS	JOIN, ON
Identical Output?	✔ Yes (1, 2, 3)	✔ Yes (1, 2, 3)	✔ Yes (1, 2, 3)

8. Performance, Space vs. Time & Exam Insights

Correlated Query Intuition

Repeatedly scans inner table for every outer row. For m outer rows and n inner rows, conceptual comparisons scale as:

$m \times n$ Comparisons

JOIN Intuition

Creates a combined relation in memory. May require initial buffer space, but reduces repeated I/O scanning overhead.

Optimized I/O Access

⚠ Critical Exam Clarification on Performance

Do **NOT** memorize that "JOIN is always faster than Correlated Subquery". Actual runtime performance depends heavily on the DBMS query optimizer, available indexes, database statistics, and execution plans.

9. Final Exam Cheat Sheet

🏆 Golden Rules

- **Non-Correlated:** Inner → Outer
- **Correlated:** Outer → Inner
- **JOIN:** Tables → Condition

⚠ Top Exam Traps

- Never assume same result = same execution.
- EXISTS checks boolean row presence.
- Use IN for multi-row subqueries.

Quick Summary Flow:

Non-Correlated: Run Inner → Output Set → Filter Outer

Correlated: Take Outer Row → Test Inner Condition → Accept/Reject

JOIN: Merge Relations → Apply On/Where Filter → Return Matches